HostelFix: Work Plan

Project: HostelFix, a hostel maintenance ticket API **Stack:** TypeScript, Node.js, Express, MongoDB, Zod, Jest
Time available: about 8 to 15 hours a week, part-time

What I will deliver each week

Week 1: Design and build

- Idea one-pager, this work plan, and a public repo with a starter README and .env.example
- **1.1 Data model and API design:** ERD, written API spec and design decisions in /docs
- **1.2 Core CRUD:** create, get, list (pagination, filters, sorting), update and delete for tickets, with validation and one consistent error format
- Seed script with dummy data, a Postman collection with success and error cases, and the API deployed (MongoDB Atlas and Render)
- 5-minute mentor presentation and a week-1 tag

Week 2: Auth and robustness

- Respond to Week 1 feedback, one line per item
- **2.1 Authentication and authorisation:** register and login, hashed passwords, access and refresh tokens, protected routes, and roles (student, staff, admin) with ownership checks
- **2.2 Relationships, validation and errors:** status transition rules and history, assignment, comments, and edge cases such as a missing room or user
- Issue log with 5 or more entries, an updated Postman collection and a week-2 tag

Week 3: Quality and delivery

- **3.1 Testing:** automated tests for main flows, auth and error cases, run with one command, plus a coverage note on what is and is not tested
- **3.2 Documentation, security and deploy:** API docs, setup guide, security and performance notes (validation, rate limiting, indexes, pagination) and a final deploy
- Final package: walkthrough video, reflection, declaration and a week-3 tag

Top 3 risks

Risk	What I will do about it
Time and scope: three part-time weeks is tight, and extra features could crowd out the core	Build the ticket lifecycle and auth first. Treat comments and recurring-fault reporting as the first things to cut
Authorisation mistakes: a student reading or changing another student's ticket, or changing status	Put ownership and role checks in middleware, and write tests for the forbidden cases
Free hosting and database setup: the live API sleeps when idle, and a wrong connection string or env variable can break it	Deploy in Week 1, note the sleep behaviour in the README, and re-test the live URL before each submission